

Reinforcement Learning — Teaching Notes

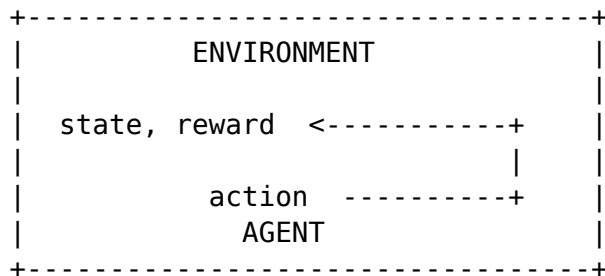
Based on Aymen Tlili's Colab experiments with Gymnasium & Stable-Baselines3

Table of Contents

1. What is Reinforcement Learning?
 2. Core Concepts
 3. The Markov Decision Process (MDP)
 4. Reward, Return, and Value Functions
 5. Policies
 6. The Bellman Equations
 7. Key Algorithm Families
 - 7.1 Q-Learning & DQN
 - 7.2 Policy Gradient & REINFORCE
 - 7.3 Actor-Critic (A2C / A3C)
 - 7.4 Proximal Policy Optimization (PPO)
 8. Custom Environments with Gymnasium
 9. Stable-Baselines3 in Practice
 10. Case Study: Gateway Selection in Sensor Networks
 11. Case Study: Link Duration Prediction
 12. Comparing PPO, A2C, and DQN
 13. Common Pitfalls and Debugging Tips
 14. Further Reading
-

1. What is Reinforcement Learning?

Reinforcement Learning (RL) is a paradigm of machine learning where an **agent** learns to make sequential decisions by interacting with an **environment**. Unlike supervised learning, RL receives no labelled data — it learns purely from **trial and error**, guided by scalar **reward** signals.



Key difference from other ML paradigms:

Paradigm	Input	Feedback
Supervised	Labelled (x, y) pairs	Error on ground truth
Unsupervised	Unlabelled data x	Structure / reconstruction
Reinforcement	State s from environment	Scalar reward r

2. Core Concepts

Term	Symbol	Definition
Agent	—	The decision-maker (neural network or table)
Environment	—	The world the agent lives in
State	$s \in \mathcal{S}$	Full description of the world at time t
Observation	$o \in \mathcal{O}$	What the agent actually sees (may be partial)
Action	$a \in \mathcal{A}$	Choice the agent makes
Reward	$r \in \mathbb{R}$	Scalar feedback after an action
Episode	—	One complete run from start to terminal state
Trajectory	τ	Sequence $(s_0, a_0, r_0, s_1, a_1, r_1, \dots)$
Discount factor	$\gamma \in [0, 1)$	How much to value future rewards
Policy	$\pi(a s)$	Probability of taking action a in state s

3. The Markov Decision Process (MDP)

An MDP is the mathematical framework underlying almost all RL problems. It is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$ — state space
- $\mathcal{A}$ — action space
- $\mathcal{P}(s'|s, a)$ — transition probability (how likely state s' follows after taking a in s)
- $\mathcal{R}(s, a, s')$ — reward function
- γ — discount factor

The Markov property: The future is independent of the past given the present state:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$$

This is what makes the problem tractable. The entire history doesn't need to be stored.

4. Reward, Return, and Value Functions

Reward

The immediate scalar signal r_t received after transition $(s_t, a_t) \rightarrow s_{t+1}$.

Return (Discounted Cumulative Reward)

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

γ close to 1 $\rightarrow$ agent is far-sighted.

γ close to 0 $\rightarrow$ agent is myopic (cares only about immediate reward).

State-Value Function $V^\pi(s)$

Expected return starting from s , following policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid s_t = s]$$

Action-Value Function $Q^\pi(s, a)$

Expected return starting from s , taking action a , then following π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid s_t = s, a_t = a]$$

Advantage Function $A^\pi(s, a)$

How much better is action a compared to the average?

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

5. Policies

Deterministic Policy

$$a = \mu(s)$$

Maps each state directly to an action.

Stochastic Policy

$$a \sim \pi(\cdot \mid s)$$

Outputs a probability distribution over actions.

Optimal Policy

The policy π^* that maximises expected cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_\pi [G_0]$$

6. The Bellman Equations

These are recursive consistency conditions that all value functions must satisfy.

Bellman Expectation (for policy π)

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')]$$

$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}(s'|s, a) \left[\mathcal{R}(s, a, s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a') \right]$$

Bellman Optimality

$$V^*(s) = \max_a \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^*(s')]$$

$$Q^*(s, a) = \sum_{s'} \mathcal{P}(s'|s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

7. Key Algorithm Families

7.1 Q-Learning & DQN

Q-Learning (tabular) — model-free, off-policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

The term in brackets is the **TD error (temporal difference error)** — the difference between the current estimate and the bootstrapped target.

DQN (Deep Q-Network) replaces the Q-table with a neural network $Q(s, a; \theta)$ and adds two stabilisation tricks: 1. **Experience Replay** — store transitions in a buffer, sample randomly to break correlations 2. **Target Network** — a slowly-updated copy of the network used to compute stable targets

$$\text{Loss} = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

Action space: DQN requires a **discrete** action space.

7.2 Policy Gradient & REINFORCE

Instead of learning a value function, directly optimise the policy parameters θ by gradient ascent on expected return:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi [G_t \nabla_\theta \log \pi_\theta(a_t | s_t)]$$

REINFORCE is the simplest instance: use full Monte Carlo returns G_t as the weight.

Intuition: If a trajectory got high return $\rightarrow$ increase the probability of those actions. If low $\rightarrow$ decrease.

Problem: High variance, slow convergence.

7.3 Actor-Critic (A2C / A3C)

Combines value-based and policy-gradient methods:

- **Actor** $\pi_\theta(a|s)$ — the policy (produces actions)
- **Critic** $V_\phi(s)$ — estimates state values (evaluates actions)

The critic reduces variance by replacing G_t with the **advantage**:

$$\nabla_\theta J(\theta) = \mathbb{E} [A(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

A2C = synchronous version. **A3C** = asynchronous (parallel workers).

Critic loss:

$$\mathcal{L}_{\text{critic}} = \mathbb{E} [(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))^2]$$

7.4 Proximal Policy Optimization (PPO)

PPO is the most widely used algorithm in practice. It solves the **step-size problem** in policy gradient: if the policy update is too large, performance can collapse.

Core idea: Clip the policy ratio to prevent large updates.

Define the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

PPO-Clip objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

The clip prevents r_t from going too far from 1, regardless of the advantage sign.

Total loss:

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{CLIP}} - c_1 \mathcal{L}^{\text{VF}} + c_2 \mathcal{H}[\pi_\theta]$$

Where $\mathcal{H}$ is an entropy bonus to encourage exploration.

Algorithm	Action Space	On/Off Policy	Memory	Notes
DQN	Discrete	Off	Replay buffer	Best for discrete, large action sets
A2C	Discrete + Continuous	On	None	Fast, lower performance ceiling
PPO	Discrete + Continuous	On (approx)	None	Best default choice

8. Custom Environments with Gymnasium

Gymnasium (successor to OpenAI Gym) provides the standard interface for RL environments.

Minimum required interface

```
import gymnasium as gym
from gymnasium.spaces import Box, Discrete
import numpy as np

class MyEnv(gym.Env):
    def __init__(self):
        super().__init__()
        # Define spaces – MUST be set before anything else
        self.observation_space = Box(low=0, high=100, shape=(3,), dtype=np.float32)
        self.action_space = Discrete(5)          # or Box for continuous

    def reset(self, seed=None):
        # Return (initial_observation, info_dict)
        obs = self.observation_space.sample()
        return obs, {}

    def step(self, action):
        # Apply action, compute next state
        obs = ...          # np.ndarray matching observation_space
        reward = ...        # float
        terminated = ...    # bool – natural episode end (e.g. goal reached)
        truncated = ...     # bool – artificial limit (e.g. max steps)
        info = {}           # optional diagnostics dict
        return obs, reward, terminated, truncated, info

    def render(self):        # optional
        pass
```

Common space types

```
from gymnasium.spaces import Box, Discrete, MultiDiscrete, MultiBinary, Tuple, Dict

# Continuous box: e.g. position in 3D
Box(low=0.0, high=100.0, shape=(3,), dtype=np.float32)

# Discrete: integer in {0, 1, ..., n-1}
Discrete(n=5)

# Multi-discrete: vector of independent discrete vars
MultiDiscrete([3, 4, 2])
```

□ Gymnasium vs Gym

Old gym (OpenAI) step() returns (obs, reward, done, info).

New gymnasium step() returns (obs, reward, terminated, truncated, info).

Stable-Baselines3 v2+ requires gymnasium. Use the compatibility wrapper if needed:

```
# Wraps old gym env to gymnasium interface
import shimmy
```

9. Stable-Baselines3 in Practice

Stable-Baselines3 (SB3) provides reliable, well-tested implementations of PPO, A2C, DQN, SAC, TD3.

Basic training loop

```
from stable_baselines3 import PPO, A2C, DQN
from stable_baselines3.common.evaluation import evaluate_policy
import torch

env = MyEnv()

# Custom network architecture
policy_kwargs = dict(
    activation_fn=torch.nn.ReLU,
    net_arch=[128, 64, 32, 16]  # shared layers for actor and critic
)

agent = PPO(
    "MlpPolicy",          # MlpPolicy for vector obs, CnnPolicy for images
    env,
    learning_rate=1e-4,
    policy_kwargs=policy_kwargs,
    verbose=1
)

agent.learn(total_timesteps=100_000)

# Evaluate
mean_reward, std_reward = evaluate_policy(agent, env, n_eval_episodes=10)
print(f"Mean: {mean_reward:.2f} ± {std_reward:.2f}")
```

Inference

```
obs, _ = env.reset()
for _ in range(1000):
    action, _ = agent.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(action)
```

```
if terminated or truncated:
    obs, _ = env.reset()
```

Choosing between algorithms

Problem has discrete actions?

- Yes: DQN or PPO/A2C with Discrete space
- No (continuous): PPO, A2C, SAC, TD3

Small action space, simple env?

- DQN (simple, stable)

Complex env, want reliable default?

- PPO (gold standard in practice)

Fast training, parallelism?

- A2C (simpler updates, scales with more envs)

Off-policy, sample efficiency matters?

- SAC (continuous), DQN (discrete)

10. Case Study: Gateway Selection in Sensor Networks

From RL_NEW.ipynb

Problem

A drone (gateway) needs to select, at each timestep, which of N base stations to connect to. The goal is to **maximise channel quality (CQI) while balancing load** across gateways.

State space

$$s = [x_1, \dots, x_{N+2}, y_1, \dots, y_{N+2}, z_1, \dots, z_{N+2}, \text{CQI}_1, \dots, \text{CQI}_N, \text{load}_1, \dots, \text{load}_N]$$

Shape: $(3 \cdot (N+2) + 2 \cdot N,)$ — a flat vector combining positions, signal qualities, and loads.

Action space

Discrete(N) — select one of N gateways.

Reward function

$$r = \frac{1}{2} \left(\frac{\text{CQI}_{\text{chosen}}}{\max_i \text{CQI}_i} + \frac{(\sum_i L_i)^2}{N \sum_i L_i^2} \right)$$

- First term: **CQI reward** — prefer the gateway with best signal quality
- Second term: **Jain's Fairness Index** — penalise unequal load distribution

CQI model

$$\text{CQI}_i = -\frac{15}{d_{\max}} \cdot d_i + 15$$

where d_i is the shortest distance from gateway i to either base station, $d_{\max} = \sqrt{30000}$.

Code highlights

```
# From Environment.step()
cqi_reward = mycqi / max(self.cqis)

sq_sm = sum(self.current_loads)**2
sm_sq = sum([x**2 for x in self.current_loads])
load_reward = sq_sm / (self.ng * sm_sq) # Jain's Fairness Index

reward = (cqi_reward + load_reward) / 2
```

Training setup

```
N_GATEWAYS = 5
TRAINING_ITERATIONS = 10000
LR = 0.01

policy_kwargs = dict(activation_fn=torch.nn.ReLU, net_arch=[128, 64, 32, 16])

for agent_name in ["PP0", "A2C", "DQN"]:
    agent_class = get_agent(agent_name)
    agent = agent_class("MlpPolicy", env, learning_rate=LR, policy_kwargs=policy_kwargs)
    agent.learn(total_timesteps=TRAINING_ITERATIONS)
```

11. Case Study: Link Duration Prediction

From RL.ipynb

Problem

Predict, at each timestep, how many steps a moving gateway will remain within radio range of a stationary sensor — a **continuous action** problem.

State space

A sliding window of 3 consecutive 3D positions of the gateway: shape (3, 3).

Action space

Box(low=0, high=max_steps, shape=(1,), dtype=np.int32) — predicted remaining duration.

Reward

$$r = -\frac{|a - d_{\text{true}}|}{d_{\text{true}}}$$

MAPE-style: normalised prediction error (negative, so maximising reward → minimising MAPE).

Key insight

DQN cannot be used here because the action space is continuous (Box). Only PPO or A2C work.

This FAILS with DQN:

```
agent = DQN("MlpPolicy", vec_env, ...)
```

AssertionError: DQN only supports Discrete action spaces

12. Comparing PPO, A2C, and DQN

Property	PPO	A2C	DQN
Action space	Discrete + Continuous	Discrete + Continuous	Discrete only
On/Off-policy	On-policy	On-policy	Off-policy
Sample efficiency	Medium	Lower	Higher
Stability	High (clipping)	Medium	Medium
Training speed	Medium	Fast	Slow (replay)
GPU benefit	Low (MLP)	Low (MLP)	Low (MLP)
Best for	General purpose	Fast prototyping	Discrete, off-policy

From the gateway experiment results: - All three converge but at different rates - PPO tends to produce the smoothest reward curves - A2C is fastest to train per timestep - DQN requires more total timesteps due to replay

13. Common Pitfalls and Debugging Tips

1. Wrong API version (Gym vs Gymnasium)

Old gym – step returns 4 values

```
obs, reward, done, info = env.step(action)
```

Gymnasium – step returns 5 values

```
obs, reward, terminated, truncated, info = env.step(action)
```

SB3 v2 wraps old gym envs automatically but warns you.

2. DQN with continuous action space

DQN only supports Discrete. Use PPO or A2C for Box action spaces.

3. Observation dtype mismatch

Spaces declared as `dtype=np.uint8` but returning float arrays → silent errors.
Always match the observation dtype in `step()` and `reset()` to what you declared.

4. Reward scale

Large rewards (>1000) or tiny rewards (<0.001) can cause instability.
Normalise rewards or use `VecNormalize` wrapper.

5. Episode length

Very long episodes make on-policy algorithms (PPO, A2C) slow.
Truncate at a maximum step count and set `truncated=True`.

6. GPU usage with MLP policies

Warning: PPO is primarily intended to run on CPU for MLP policies

This is expected — for vector observation spaces, GPU overhead outweighs the compute benefit.

Pass `device='cpu'` explicitly.

7. Hyperparameter sensitivity

RL is notoriously sensitive. Default starting points:

```
PPO(learning_rate=3e-4, n_steps=2048, batch_size=64, n_epochs=10, gamma=0.99, clip_range=
A2C(learning_rate=7e-4, n_steps=5, gamma=0.99, gae_lambda=1.0)
DQN(learning_rate=1e-4, buffer_size=100_000, learning_starts=1000, batch_size=32, gamma=0
```

14. Further Reading

Resource	Focus
Sutton & Barto — <i>Reinforcement Learning: An Introduction</i> (free online)	Theory foundation, Bellman, TD, Q-learning
Spinning Up (OpenAI)	Policy gradients, PPO, SAC — code + math
Stable-Baselines3 docs	API reference, examples
CleanRL	Single-file, readable RL implementations
Gymnasium docs	Environment API, built-in envs
David Silver's RL Course (UCL/DeepMind)	Full lecture series

Notes generated from Aymen Tlili's RL Colab notebooks (RL.ipynb, RL_NEW.ipynb, RL_OLD.ipynb)